

MicroPython Code Review Report

Analyzed File: balloon2026_1_1.py | Project: Stratospheric Balloon Payload

Executive Summary: The overall script exhibits a sound functional architecture. However, for a stratospheric flight (extreme low temperatures and power instability hazards), **4 critical program crash risks** were identified. This document details these anomalies and provides optimized, flight-ready code.

1. Critical Risks & Runtime Errors

Issue / Line	In-Flight Impact	Recommended Fix
Missing DS18B20 Check Lines 50 / 87	If sensor is disconnected or freezes, id[0] raises an IndexError and crashes the script immediately.	Check if id list is non-empty before reading and wrap in a try...except block.
Duplicated CSV Header Line 74	In append ('a') mode, any in-flight MCU reboot rewrites the header row in the middle of data.	Check file existence via os.stat() before writing the header.
Busy-Wait Loop Line 102	The while pass loop burns 100% CPU for 4 seconds, increasing self-heating and draining battery faster.	Replace with dynamic time.sleep_ms() to put the CPU into low-power idle mode.
Cumulative Time Drift Line 105	Resetting start_time every cycle accumulates execution delays from I2C/Flash I/O operations.	Keep the original mission start_time to compute true elapsed time accurately.

2. Engineering Improvements

- **Fault Tolerance:** Wrapping each sensor read in try...except logs NC (Not Connected) rather than killing the entire mission if an I2C bus temporarily freezes.
- **Power Optimization:** Switching to passive sleep (sleep_ms) during sampling intervals extends battery life on the 14500 cell.
- **CSV Sanitization:** Removing formatting spaces in text strings ensures seamless automated numeric parsing in spreadsheet tools.

3. Recommended MicroPython Code (balloon2026_1_1_fixed.py)

```
# File : balloon2026_1_1_fixed.py
import machine, onewire, ds18x20, time, ina219, lps22hb, os
from machine import Pin

PERIOD, DURATION = 5000, 180
FILENAME = "balloon_data.csv"
CORR_PRESS = 1033 / 991
SAMPLES = DURATION // (PERIOD // 1000)

hb = Pin(25, Pin.OUT)
hb.off()

# Initializations
ow = onewire.OneWire(Pin(22))
ds = ds18x20.DS18X20(ow)
ds_roms = ds.scan()

lps = lps22hb.LPS22HB()
bat_mon = ina219.INA219(addr=0x43)

# Verification / creation of CSV file
try:
    os.stat(FILENAME)
except OSError:
    with open(FILENAME, "w") as f:
        f.write("Time;T_out;T_in;pressure;voltage;current\n")

start_mission_time = time.ticks_ms()
count = 0
```

```

while count < SAMPLES:
    cycle_start = time.ticks_ms()
    hb.on(); time.sleep_ms(50); hb.off()
    elapsed_time = time.ticks_diff(cycle_start, start_mission_time) // 1000

    # 1. DS18B20 (Outside temperature)
    str_T_out = "NC"
    if ds_roms:
        try:
            ds.convert_temp()
            time.sleep_ms(750)
            str_T_out = f"{ds.read_temp(ds_roms[0]):.1f}".replace('.', ',')
        except: pass

    # 2. LPS22HB & INA219 (Pressure, Inside Temp, Voltage, Current)
    try:
        str_pressure = f"{lps.pressure * CORR_PRESS:.1f}".replace('.', ',')
        str_T_in = f"{lps.temperature:.1f}".replace('.', ',')
        str_voltage = f"{bat_mon.getBusVoltage_V():.3f}".replace('.', ',')
        str_current = f"{bat_mon.getCurrent_mA():.3f}".replace('.', ',')
    except:
        str_pressure = str_T_in = str_voltage = str_current = "NC"

    # 3. Write to Flash memory
    try:
        with open(FILENAME, "a") as f:
            f.write(f"{elapsed_time};{str_T_out};{str_T_in};{str_pressure};{str_voltage};{str_
current}\n")
    except: pass

    count += 1

    # 4. Low-power idle sleep
    remaining_sleep = PERIOD - time.ticks_diff(time.ticks_ms(), cycle_start)
    if remaining_sleep > 0:
        time.sleep_ms(remaining_sleep)

hb.on(); time.sleep_ms(2000); hb.off()

```